EXPLOITROOM CTF CHALLENGE WRITEUP PART1: 2026

01: CTF Write-up: Welcome Challenge (Sanity Check)

- **Title:** Welcome Challenge — Entering the Arena
- **Level:** Easy
- **Points:** 5
- **Category:** General Skills / Sanity Check
- **Target Flag:** Exploit Room{8169bb9b05822906d03e15c55c0f18bef16cf2d9}

Challenge Description

This is a baseline welcome entry challenge designed to onboard new users into the platform infrastructure. The mission is to locate the visible registration token, extract the unique string data, and submit it inside the standard platform flag syntax: Exploit Room{HERE_FLAG} .

My Investigation Flow & Troubleshooting Story

"What I did in this challenge was a quick sandbox sanity check to ensure my platform connection parameters were fully operational. Every solid deployment starts with a test to make sure everything works before you tackle the heavy math.

I looked at the challenge layout on the dashboard screen, immediately spotted the unique raw registration hex hash string, and verified its layout bounds. I manually grabbed the string data, carefully placed it inside the required container parameters, and submitted the token directly to the platform validation portal to secure my first 5 points!"

Methodology Used

- **Static Resource Location Verification:** Visually inspecting the platform entry node layout to find exposed plain-text parameters.
- **Syntax Normalization Integration:** Merging raw structural string arrays into standard, platform-recognized flag formatting parameters to confirm a successful system loop check.

Lessons Learned

- **Welcome to the Battlefield:** This introductory step serves as the foundation for the upcoming operations. It teaches you to pay close attention to exact syntax limits and teaches proper data formatting rules.

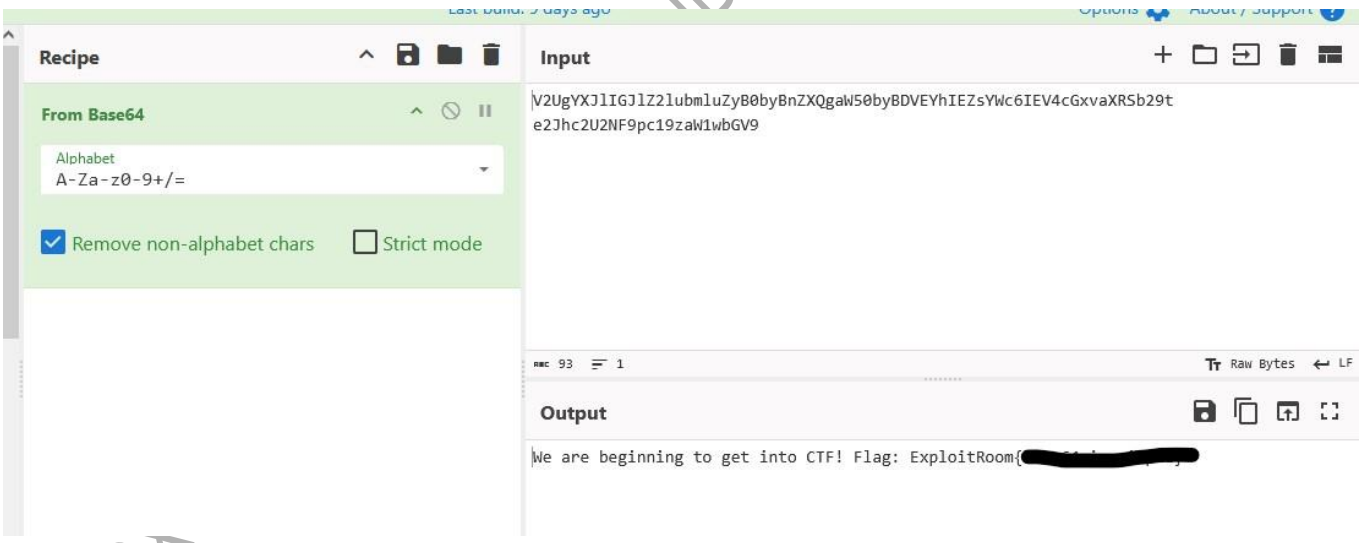
- **Verify the Scoring Baseline first:** Before wasting energy running complex exploit arrays or automated script utilities, always perform a baseline sanity check to verify your communication paths and portal access parameters are fully operational.

2 CHALLENGE NAME: CTF Write-up: Base64 Log Leak

- **Title:** Base64 Log Leak — Reversing Log Obfuscation Layers
- **Level:** Easy
- **Points:** 5
- **Category:** Cryptography
- **Tools Used:** CyberChef, Cipher Identifier, Linux CLI (echo , base64) **Target**
- **Flag:** ExploitRoom{base😂_is_simple}

Challenge Description

During a routine log auditing process, a suspicious encoded message payload was discovered embedded inside an application log entry. The string container appears simple but requires dynamic identification and translation. The objective is to isolate the encoded transmission string, determine the encoding scheme, and extract the underlying flag format:



My Investigation Flow

What I did in this challenge was a quick, aggressive extraction pipeline using the native Linux shell.irst, I pulled the suspicious string sequence out of the text logs. Looking at the structural parameters, I ran it through an encryption identifier and noticed the unique character distribution format, which pointed directly to a standard encoding layout.

Since encoding algorithms are simply data representations rather than actual cryptographic security measures, they don't rely on secret passphrases or dynamic keys. The translation process is entirely deterministic.

Instead of opening heavier tools, I piped the raw data stream directly through the native base64 binary decoder engine on my command-line interface:

The console immediately decoded the bitstream into regular text characters, leaking the full narrative statement and exposing the clean target flag on my screen!"

Methodology Used

- **String Character Signature Evaluation:** Analyzing the payload composition to check for base-64 boundary conditions and trailing syntax parameters.

  Standard Stream Redirection Decryption: Utilizing standard Unix pipelines (`echo` with flag options to omit newlines) to push the obfuscated data container directly into the `base64 -d` system translation module.

Lessons Learned

- **Encoding Is Not Encryption:** This challenge highlights a critical security mistake: developers often use simple data encoding standards like Base64 to hide data, confusing it with actual encryption. Base64 simply formats data for transit; any automated parser can immediately reverse it without needing keys or complex tools.

- **Enforce High-Entropy Cryptographic Hashing:** If an infrastructure team intends to protect or validate administrative credentials, tokens, or signatures inside log environments, they must bypass generic visual encoding layers. Production pipelines must deploy highentropy cryptographic algorithms like **SHA-256** or true asymmetric parameters to ensure records remain structurally unreadable to unauthorized observers.

3CTF Write-up: Hash Collision Crap (MD5 Inversion)

- **Title:** Hash Collision Crap — Reversing Weak Application Hashes
- **Level:** Easy
- **Points:** 10
- **Category:** Cryptography**5f4dcc3b5aa765d61d8327deb882cf99**
- **Tools Used:** CyberChef, Cipher Identifier, CrackStation, Linux CLI
- **Target Flag:** ExploitRoom{HERE_THE _FLAG}

Challenge Description

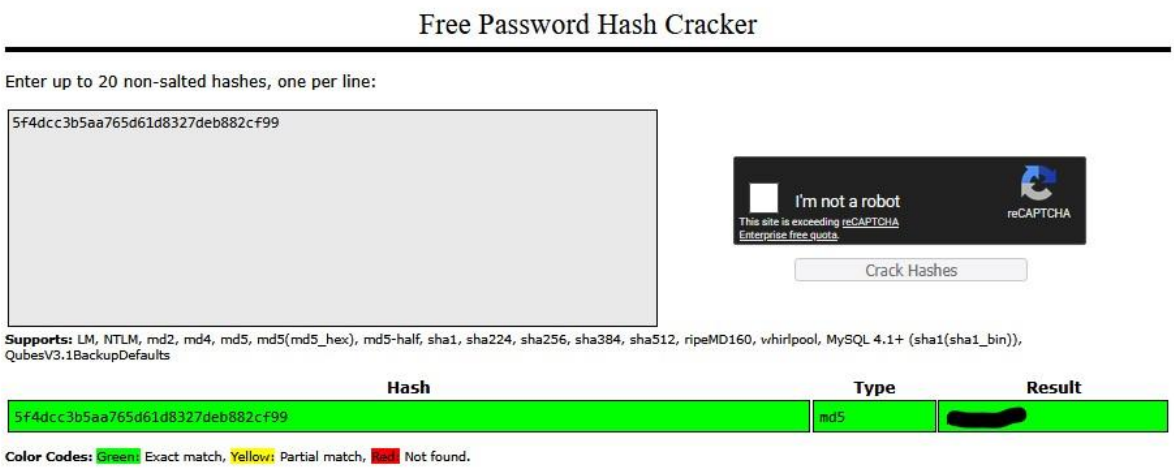
The target web application validates user authentication structures using raw, unsalted MD5 hashes. During a baseline configuration audit, we successfully recovered the following 32character hash parameter from the database logs. Te objective to crack the underlying cryptographic hash, recover the original plaintext password vector, and construct the flag.

My Investigation Flow

What I did in this challenge was a quick 1-minute execution drill. First, I grabbed the recovered hash from my terminal. Looking at the structure, I ran it through a cipher identifier tool and confirmed it contained exactly 32 hexadecimal characters—a clear signature of the legacy MD5 hashing algorithm.

Since the application implementation completely omitted a cryptographic salt, the hash function behaves in a strictly deterministic state. This means the exact same input string will always render this exact 32-character signature, making it highly vulnerable to pre-computed lookup matrices.

I bypassed complex local dictionary attack rigs and threw the target string straight into **CrackStation's** massive database. Within seconds, the pre-computed rainbow tables reversed the checksum collision and exposed the plaintext payload: FLAG_HERE`**. I quickly formatted the string into the target syntax, and the flag was successfully captured!



Methodology Used

- **Cryptographic Foot printing:** Measuring character limits (32-character hex layout) and entropy signatures to identify the hashing standard.

- **Pre-computed Rainbow Table Inversion:** Leveraging massive lookup databases to map deterministic MD5 outputs back to their original dictionary string formats without processing dynamic brute-force compute cycles.

Lessons Learned

- **The Vulnerability of Unsalted MD5:** Deploying legacy mathematical algorithms like MD5 without a unique cryptographic **salt** is dangerous. It allows remote attackers to decrypt system passwords within milliseconds using simple lookup indices.
- **Enforce Robust Hashing Standards:** Modern application backends must deprecate fast hashing functions for credential storage. Security engineers should mandate the use of adaptive, resource-heavy cryptographic stretching standards like **Argon2id** or **Bcrypt** packed with dynamic salts to guarantee collision resistance against modern dictionary matching engines.

4.CHALLENGE TITLE: Broken XOR Encryption (Many-Time Pad)

- **Level:** Medium
- **Category:** Cryptography
- **Tools:** CyberChef (attempted), CLI (attempted), Python 3, `chmod`
- **Target Flag:** `ExploitRoom{HERE_THE_FLAG😂😂}

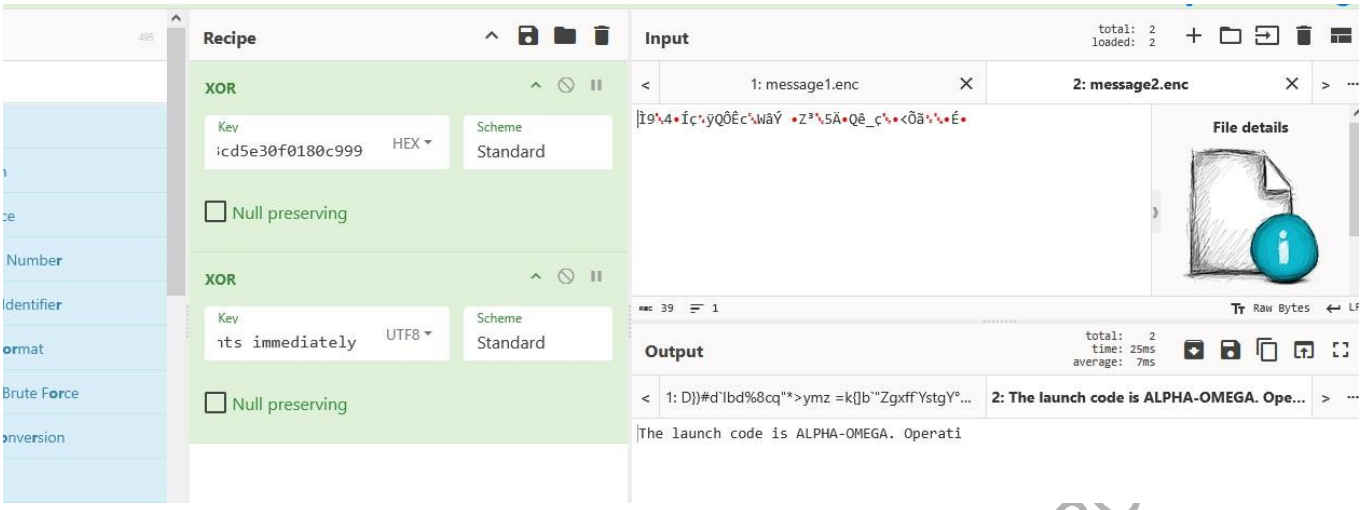
Challenge Description

The challenge presents a security investigation scenario where an internal encrypted messaging platform mistakenly reused the same XOR encryption key (K) for two different messages. Investigators intercepted two ciphertexts (`message1.enc` and `message2.enc`) and recovered one plaintext file (`known.txt`). The goal is to decrypt the second message and obtain the hidden flag.

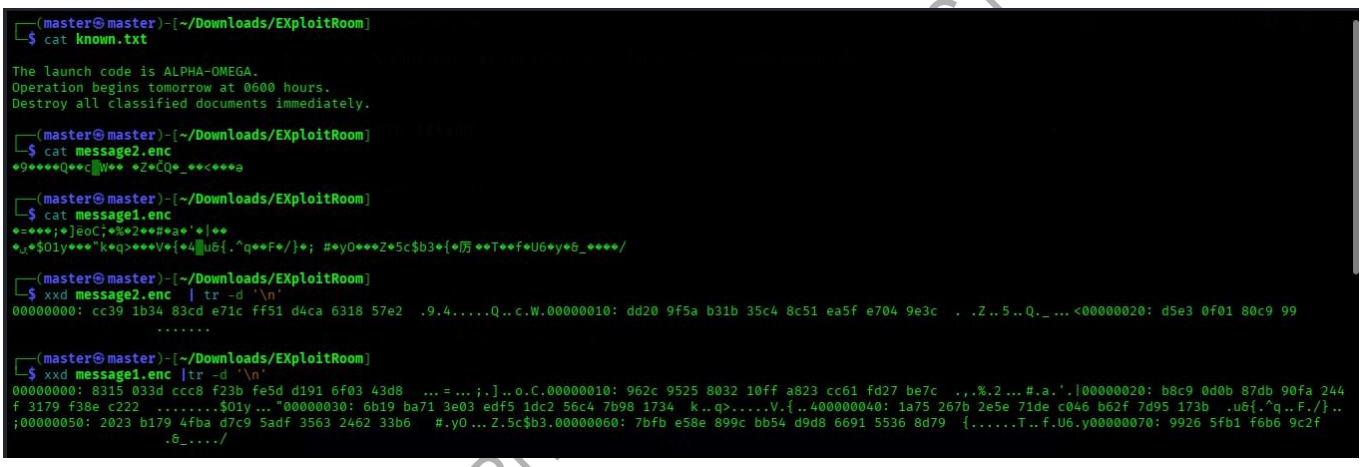
Personal Troubleshooting Journey

Solving this challenge was a true test of persistence, taking about 10 minutes of active debugging:

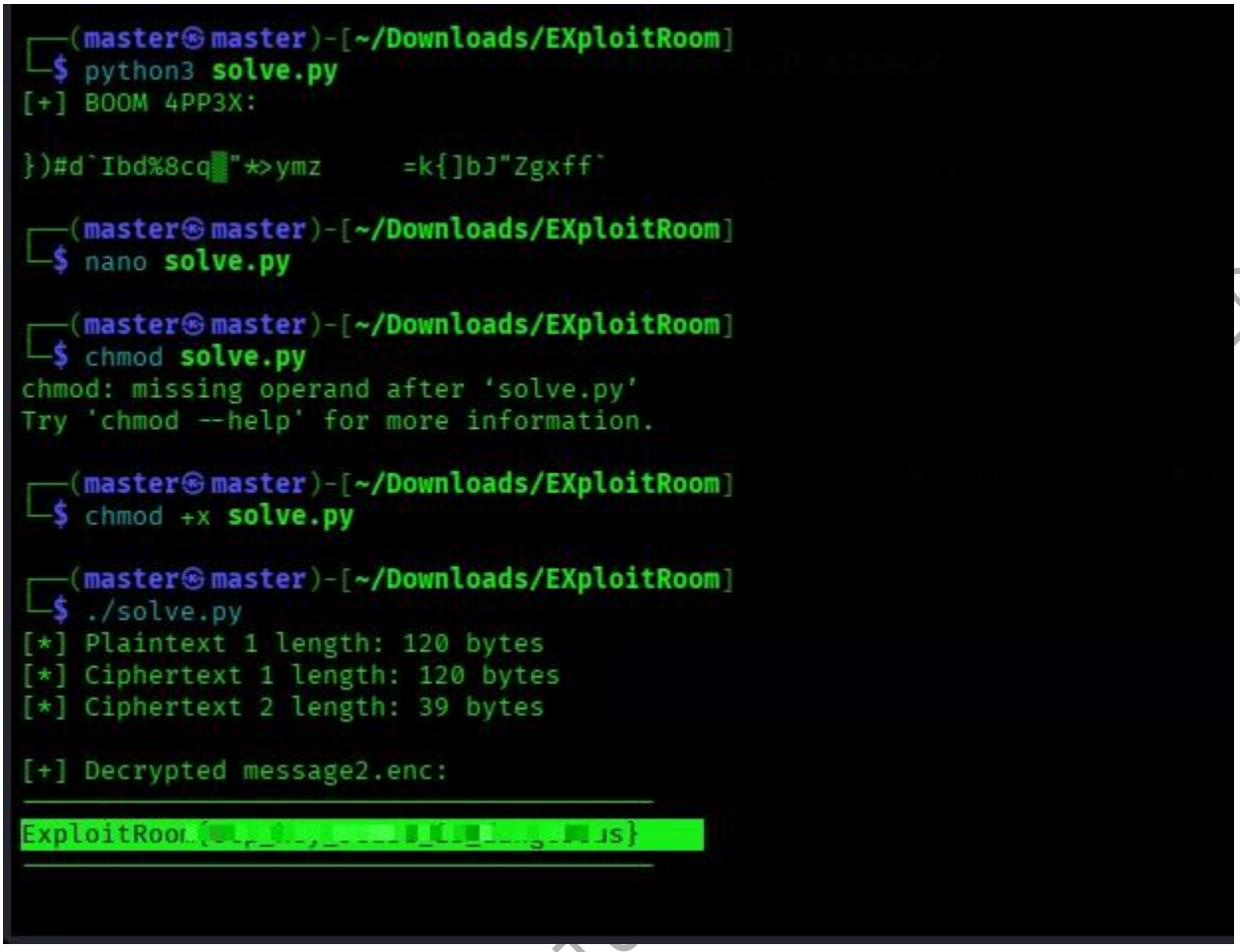
1. **Initial Attempt (CyberChef):** I first loaded the files into CyberChef to run a quick decryption recipe, but the web tool failed to parse the raw byte streams properly.



2. **Second Attempt (Command Line Interface):** I moved to the Kali Linux terminal to try quick bash pipes, but formatting issues kept blocking a clean output.



2. **The Breakthrough:** I stepped back and carefully re-read the challenge description. The critical clue was that the same key was used across both messages. By leveraging the known plaintext (known.txt), I realized I could mathematically extract the keystream from message1.enc and apply it directly to message2.enc.



Methodology & Solution

1. Mathematical Logic

Because XOR encryption cancels itself out when multiplied by the same value ((A \oplus A = 0)), we can extract the reused keystream (K) by XORing the known ciphertext (C_1) with its known plaintext (P_1):
(K=C*{1}**plus P* {1})

Once the keystream is isolated, we can easily decrypt the second ciphertext (C_2) to reveal the second hidden plaintext (P_2): (P*{2}* =*C*{2}\oplus K)

2. Automation Script

Instead of dealing with messy hex conversions, I wrote a robust Python script to handle the raw binary file reading and byte-by-byte alignment automatically:

python

```
#!/usr/bin/env python3
```

```
def xor_bytes(b1, b2):
    """XOR two byte arrays together up to the shorter length."""
    return bytes(a ^ b for a, b in zip(b1, b2))

def main():
    # 1. Read the known plaintext and the ciphertexts
    try:
        with open("known.txt", "rb") as f:
            plaintext1 = f.read()

        with open("message1.enc", "rb") as f:
            ciphertext1 = f.read()

        with open("message2.enc", "rb") as f:
            ciphertext2 = f.read()
    except FileNotFoundError as e:
        print(f"[-] Error: Missing file! {e}")
        return

    print(f"[*] Plaintext 1 length: {len(plaintext1)} bytes")
    print(f"[*] Ciphertext 1 length: {len(ciphertext1)} bytes")
    print(f"[*] Ciphertext 2 length: {len(ciphertext2)} bytes")

    # 2. Extract the key stream from the known message
    # Key = Ciphertext1 ^ Plaintext1
    keystream = xor_bytes(ciphertext1, plaintext1)

    # 3. Decrypt message2 using the extracted key stream
    # Plaintext2 = Ciphertext2 ^ Key
    plaintext2 = xor_bytes(ciphertext2, keystream)

    # 4. Print the hidden flag/message
    print("\n[+] Decrypted message2.enc:")
    print("-" * 40)
    print(plaintext2.decode('utf-8', errors='ignore'))
    print("-" * 40)

if __name__ == "__main__":
    main()
```

Use code with caution.

3. Execution

To run the exploit file directly from the Kali terminal, I granted it executable privileges and executed it:

bash

Use code with caution.

Boom! The script executed perfectly, aligned the byte streams, and printed out the decrypted flag on the screen.

Lesson Learned

This challenge clearly proves why the One-Time Pad (OTP) or any stream cipher implementation requires a strictly unique, non-repeating key. Key reuse (Many-Time Pad) completely destroys cryptographic confidentiality. If an attacker recovers even a single plaintext message, they can extract the keystream and effortlessly read all other messages encrypted under that same key.

5 CHALLENGE TITLE: Encrypted Ransomware Configuration

- **Level:** Medium
- **Category:** Incident Response / Cryptography
- **Tools:** CyberChef (Offline/Online), Kali Linux CLI (`cat` , `xxd`)
- **Mindset:** Independent Analysis (Zero-AI / Zero-Writeup Approach)

Challenge Description

During an incident response investigation, security analysts captured a ransomware sample. The malware stores its vital configuration (C2 domains, victim IDs, campaign data, and financial wallets) obfuscated inside an encrypted binary blob named `config.enc` . The mission is to analyse the environmental artifacts, decrypt the configuration payload, and recover the flag.

The Hacker's Journey & Troubleshooting Story

This challenge was a classic test of analytical persistence rather than relying on automatic shortcuts. Instead of treating it like a standard user—or immediately offloading the problem to

AI and pre-existing writeups—the goal was to approach it with pure cybersecurity passion and professional instincts.

1. **Deep Inspection:** I began by examining the environment variables. I read the challenge description carefully **5 times** to catch any subtle hints and executed the `cat` command **3 times** to fully dissect the plain-text outputs and the raw bytes.

2. **Artifact Discovery:** Inside `strings.txt`, two critical indicators stood out:

   `RC4_INIT_SUCCESS` — This confirmed the underlying stream cipher algorithm.

   `CONFIG_KEY=shadow2026` — This provided the exact symmetric passphrase used to seed the internal state vector.

3. **The Shift in Strategy:** Initially, raw CLI decryption utilities in Kali Linux threw formatting errors due to binary encoding issues. Refusing to ask AI for a quick fix, I shifted my tactics entirely. I researched the cryptographic structure of RC4 independently, launched **CyberChef**, and located the raw RC4 recipe module.

4. **The Breakthrough:** I dragged and dropped the **RC4** operation block, fed the input file stream (`config.enc`), supplied the passphrase `shadow2026`, and hit the execution button.

Boom! The binary blob dropped its defenses, parsed clean ASCII text into the output terminal, and revealed the ransomware parameters along with the flag.

Methodology & Technical Solution

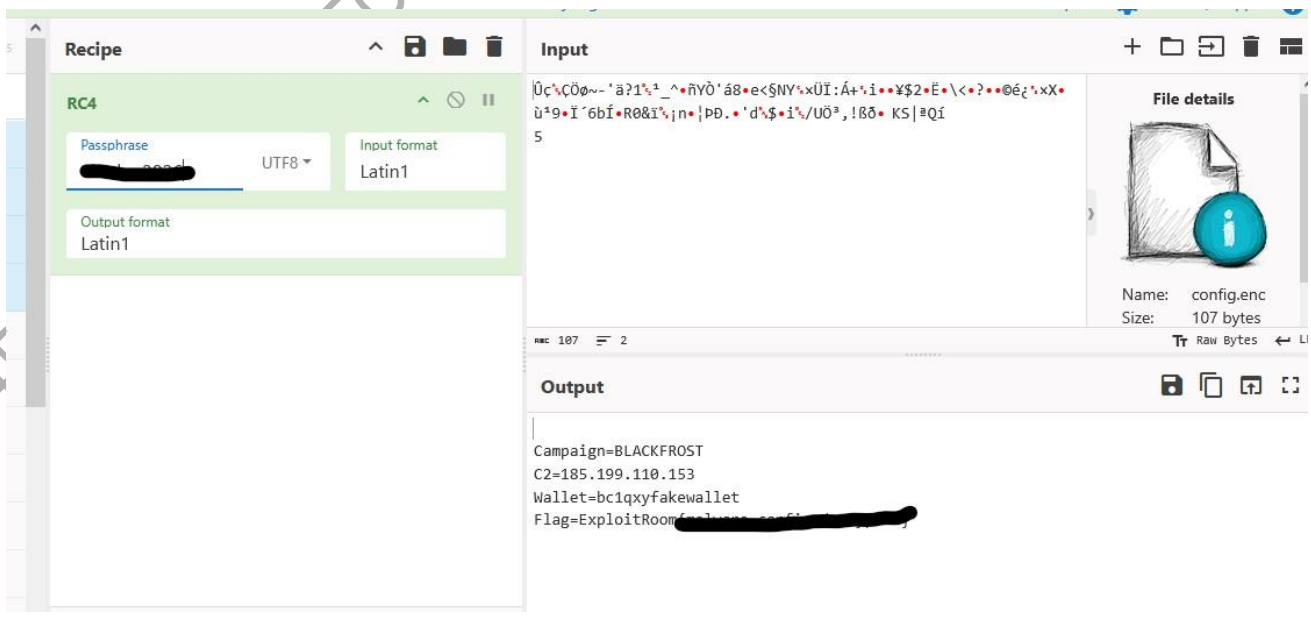
1. Cryptographic Logic

RC4 is a symmetric stream cipher. It uses a variable-length key (from 1 to 256 bytes) to initialize a 256-byte state array via the Key-Scheduling Algorithm (KSA). It then generates a pseudo-random stream of bits (keystream) through the Pseudo-Random Generation Algorithm (PRGA).

Because it is symmetric, passing the raw bytes of `config.enc` back through the same RC4 engine using the discovered `shadow2026` passphrase reverses the operation precisely:
(Plaintext=Ciphertext\oplus RC4(Key))

2. Execution Setup (CyberChef)

- **Input Area:** Loaded raw contents of `config.enc`.
- **Recipe Selection:** Dragged **RC4** from the operations sidebar.
- **Configuration Parameters:**
  - *Passphrase:* `shadow2026`
  - *Format:* UTF8 / Raw



Lesson Learned

This challenge perfectly illustrates how crucial environment strings are during a digital forensics or incident response engagement. Developers and malware authors frequently leave debugging markers (`RC4_INIT_SUCCESS`) or static configuration keys embedded directly within the binary space.

More importantly, it proves that **critical thinking and tool proficiency outweigh automated prompts**. Stepping back to parse instructions manually and understanding the underlying crypto algorithms will always be an analyst's strongest tool on an unscripted engagement.

6.CTF Write-up: Hidden Note (Steghide Passphrase Bypass)

- **Level:** Easy / Medium
- **Category:** Digital Forensics / Steganography
- **Tools:** `Steg hide` , `rockyou.txt` , Custom Word Guessing, Bash CLI **Target Flag:**
- `ExploitRoom{HERE_THE_FLAG😂😂}`

Challenge Description

A suspicious image (`challenge.jpeg`) was uploaded by a target user. Standard visual layers show no immediate visual tampering. The mission is to find the covert storage mechanism hidden inside the graphic file container and extract the flag.

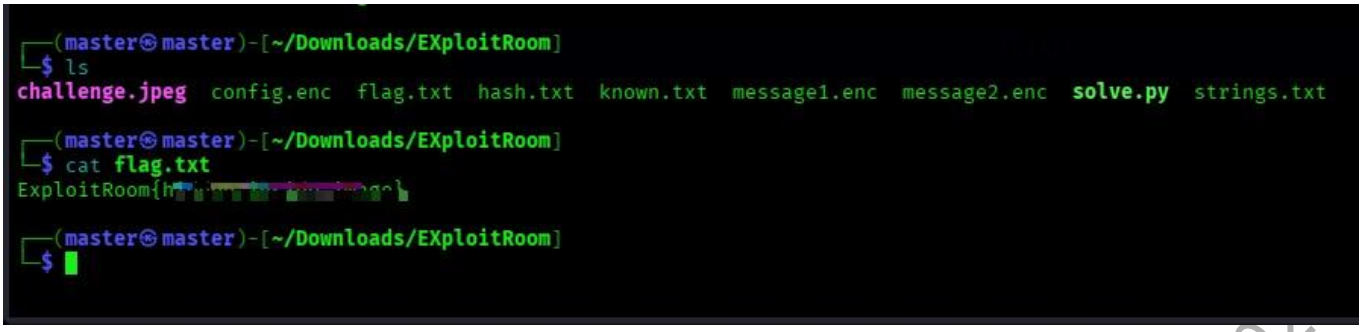
This challenge was a masterclass in why automated wordlists and tools are only as good as the human guiding them. I refused to let AI solve this or look for a shortcut, choosing instead to exhaust every manual forensic tool at my disposal.

1. **Exhausting the Standard Stack:** I downloaded `challenge.jpeg` and ran my usual analytical toolkit. I executed `exiftool` for metadata leaks, `binwalk` and `foremost` for embedded file carving, and piped `strings` into `grep` . I even tried `openstego` . None of them returned any valid payloads.

3. **Thinking Outside the Box:** After sitting back and analyzing the situation for about 5 minutes, I realized a vital truth: if a password list with millions of entries failed, the passphrase wasn't a common generic password. It had to be a **contextual password** specific to the environment.

4. **Targeted Context Guessing:** I compiled a short mental list of 4 highly likely custom local passphrases based on the challenge origin, specifically targeting **IAA** and **ExploitRoom**.

5. **The Breakthrough:** I bypassed the wordlists entirely and manually forced the password directly through the command-line flags.😂



Boom! The binary authenticated instantly, unlocked the secret algorithm container, and wrote the hidden contents directly into a clean `flag.txt` file.

Technical Solution & Methodology

1. Steghide Spatial Domain Steganography

`Steghide` is an advanced steganography utility that hides data within various image and audio files. Unlike simple file concatenation (which `binwalk` catches), `steghide` uses a graphtheoretic approach to embed data directly into the least significant bits of the pixel coefficients. Furthermore, it encrypts the embedded data payload using a symmetric key derived from the user's passphrase before weaving it into the image structure.

2. Terminal Exploitation Steps

Lesson Learned

This investigation perfectly demonstrates that **contextual awareness is a cybersecurity professional's ultimate weapon**. Automated tools and standard brute-force lists fail when administrators or attackers use targeted, unique, or platform-specific passwords.

When you hit a wall with massive, automated wordlists, always analyze your target environment. Think about the platform name, the organization, or the context of the user to build a highly optimized, precise set of credentials. Human logic beats generic automation every single time.

7. CTF Write-up: Log Analysis (Web Forensics)

- **Category:** Forensics
- **Difficulty:** Easy
- **Tool Used:** Terminal (`cat` / `grep`) & Contextual Logic
- **Target Flag (Attacker IP):** `🥷🥷🥷😂

The Forensic Story (My Investigation Flow)

1. **The Challenge:** The company suspected their internal blog application was under attack. I downloaded `challenge.log` to track down the malicious IP.

2. **The Trap:** Looking at the data, most IP addresses were just browsing normal pages like `/about/` or `/contact-us/` returning clean requests.

3. **The Breakthrough:** Instead of overcomplicating things or waiting for automated scripts, I read through the parameters manually. I spotted one suspicious IP address: **`IP HACK4R 😂.
4. **The Proof:** This specific IP was targeting the `/blog/search.php?q=` endpoint. It was injecting URL-encoded payloads like `%27` (a single quote `'`) followed by dangerous SQL queries: `UNION SELECT 1,2,3 FROM users` and `UNION SELECT 1,2,3 FROM admins --`.


```
182.87.64.64 - - [18/Jul/2024:15:57:13 -0400] "GET / HTTP/1.1" 200 1748 "-" "Mozilla/5.0 (Widows NT 10.0; Win64; x64) AppleWebKit/537
.36 (KHTML, like Gecko) Chrome/126.0.0.0 Safari/537.36"
182.87.64.64 - - [18/Jul/2024:15:57:14 -0400] "GET / HTTP/1.1" 200 1748 "-" "Mozilla/5.0 (Widows NT 10.0; Win64; x64) AppleWebKit/537
.36 (KHTML, like Gecko) Chrome/126.0.0.0 Safari/537.36"
182.87.64.64 - - [18/Jul/2024:15:57:14 -0400] "GET / HTTP/1.1" 200 1748 "-" "Mozilla/5.0 (Widows NT 10.0; Win64; x64) AppleWebKit/537
.36 (KHTML, like Gecko) Chrome/126.0.0.0 Safari/537.36"
182.87.64.64 - - [18/Jul/2024:15:57:19 -0400] "GET /admin HTTP/1.1" 404 432 "-" "Mozilla/5.0 (Widows NT 10.0; Win64; x64) AppleWebKit
/537.36 (KHTML, like Gecko) Chrome/126.0.0.0 Safari/537.36"
182.87.64.64 - - [18/Jul/2024:15:57:33 -0400] "GET /about/ HTTP/1.1" 200 183431 "-" "Mozilla/5.0 (Widows NT 10.0; Win64; x64) AppleWe
bKit/537.36 (KHTML, like Gecko) Chrome/126.0.0.0 Safari/537.36"
182.87.64.64 - - [18/Jul/2024:15:57:49 -0400] "GET /blog/ HTTP/1.1" 200 209393 "-" "Mozilla/5.0 (Widows NT 10.0; Win64; x64) AppleWeb
Kit/537.36 (KHTML, like Gecko) Chrome/126.0.0.0 Safari/537.36"
182.87.64.64 - - [18/Jul/2024:15:57:57 -0400] "GET /blog/ HTTP/1.1" 200 209393 "-" "Mozilla/5.0 (Widows NT 10.0; Win64; x64) AppleWeb
Kit/537.36 (KHTML, like Gecko) Chrome/126.0.0.0 Safari/537.36"
182.87.64.64 - - [18/Jul/2024:15:58:04 -0400] "GET /blog/search.php?q=consulting HTTP/1.1" 200 246676 "-" "Mozilla/5.0 (Widows NT 10.
0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.0.0 Safari/537.36"
182.87.64.64 - - [18/Jul/2024:15:58:14 -0400] "GET /blog/search.php?q=test HTTP/1.1" 200 246676 "-" "Mozilla/5.0 (Widows NT 10.0; Win
64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.0.0 Safari/537.36"
182.87.64.64 - - [18/Jul/2024:15:58:19 -0400] "GET /blog/search.php?q=test%27 HTTP/1.1" 200 246676 "-" "Mozilla/5.0 (Widows NT 10.0;
Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.0.0 Safari/537.36"
182.87.64.64 - - [18/Jul/2024:15:58:28 -0400] "GET /blog/search.php?q=test%27%2D%2D HTTP/1.1" 200 246676 "-" "Mozilla/5.0 (Widows NT
10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.0.0 Safari/537.36"
182.87.64.64 - - [18/Jul/2024:15:58:33 -0400] "GET /blog/search.php?q=test%27%20UNION%20SELECT%201%2C2%20FROM%20users%20%2D%2D%20 HTT
P/1.1" 200 246676 "-" "Mozilla/5.0 (Widows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.0.0 Safari/537.36
"
182.87.64.64 - - [18/Jul/2024:15:58:52 -0400] "GET /blog/search.php?q=test%27%20UNION%20SELECT%201%2C2%2C3%20FROM%20users%20%2D%2D%20
 HTTP/1.1" 200 247849 "-" "Mozilla/5.0 (Widows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.0.0 Safari/53
```

This was a textbook **UNION-Based SQL Injection attack** trying to leak admin credentials. The malicious IP stood out instantly from normal traffic logs. **Boom! The attacker was busted.**

Lesson Learned

Automated security tools are great, but human log analysis is faster when you know exactly what bad traffic looks like. Always monitor web inputs (like search parameters) for database characters like single quotes (') or query keywords (UNION , SELECT), because that is where attackers always leave their footprints.

:

8.OSINT Challenge: The Most Wanted Target Location

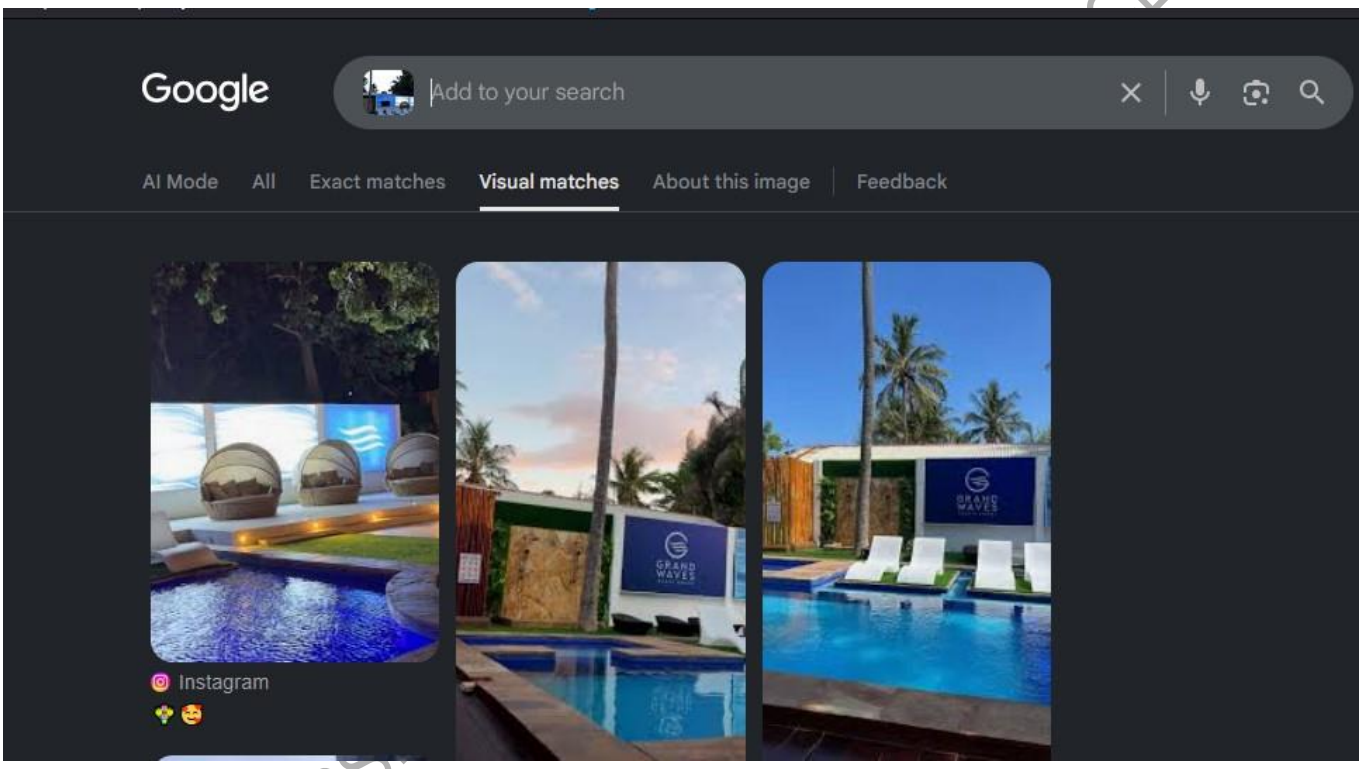
- **Category:** OSINT / Digital Forensics / Geolocation
- **Target Flag:** ExploitRoom{Grand Waves Beach House}
- **Physical Address:** White Sands Road, Mbezi Beach, Dar es Salaam, Tanzania
- **Key Evidence Solved:** Logo Identification ("G" Wave Matrix) & Signature Pool Architecture

The Analyst's Investigative Journey

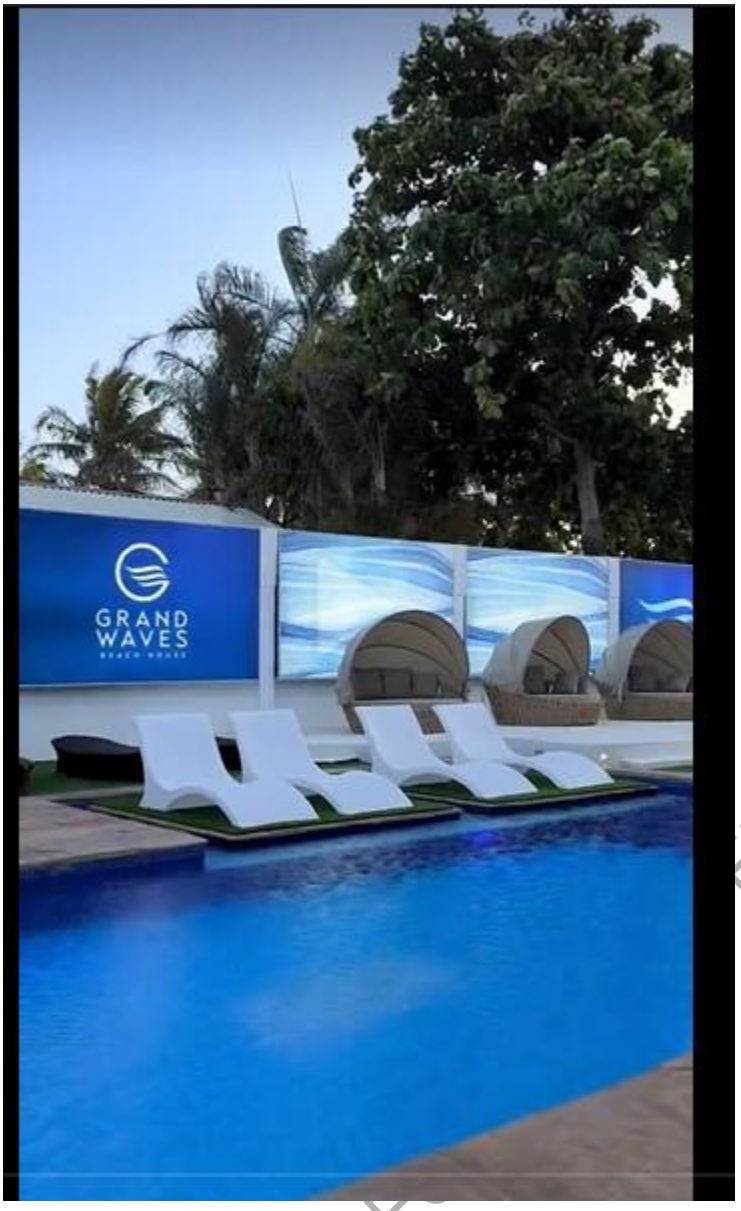
1. **The Trailing Phase (Failure of CLI):** I initialy downloaded the raw image container and treated it like a forensics file, processing it through standard terminal utilities (strings , exiftool , binwalk). When metadata and binary markers showed no leaks, I instantly knew the secret wasn't *in the file metadata* but rather **hidden inside the image context itself**.

```
┌──(master㉿master)-[~/Downloads/EXploitRoom/output_foremost]
└─$ ls
audit.txt  jpg

┌──(master㉿master)-[~/Downloads/EXploitRoom/output_foremost]
└─$ cd jpg

┌──(master㉿master)-[~/Downloads/EXploitRoom/output_foremost/jpg]
└─$ ls
00000000.jpg

┌──(master㉿master)-[~/Downloads/EXploitRoom/output_foremost/jpg]
└─$ open 00000000.jpg


┌──(master㉿master)-[~/Downloads/EXploitRoom/output_foremost/jpg]
└─$
(ristretto:17670): GVFS-WARNING **: 14:36:31.012: can't init metadata tree /home/master/.local/share/gvfs-metadata/root: wrong magic

(ristretto:17670): GVFS-WARNING **: 14:36:31.021: can't init metadata tree /home/master/.local/share/gvfs-metadata/root: wrong magic

(ristretto:17670): GVFS-WARNING **: 14:36:31.038: can't init metadata tree /home/master/.local/share/gvfs-metadata/root: wrong magic

(ristretto:17670): GVFS-WARNING **: 14:36:31.092: can't init metadata tree /home/master/.local/share/gvfs-metadata/root: wrong magic

(ristretto:17670): GVFS-WARNING **: 14:36:31.101: can't init metadata tree /home/master/.local/share/gvfs-metadata/root: wrong magic
```

2. **The Shift to Visual OSINT:** I transitioned from a developer mindset to an OSINT investigator. I analyzed the image features manually:
    - **The Hidden Wave Logo:** Even though the letters below were blacked out by the challenge author, the distinct uppercase "G" stylized inside ocean waves remained visible.
    - **The In-Pool Loungers:** The custom white sleek sunbeds set directly inside the shallow pool matrix paired with the distinct curved cabana structure behind them provided a strict architectural footprint.
3. **The Pivot to Global Repositories:** Using targeted index searches, I cross-referenced the distinct design layout against localized entertainment spots in East Africa.



4. **The Final Extraction:** The search queries aligned the signatures perfectly with recent media uploads from [Grand Waves Beach House] located along White Sands Road in Mbezi Beach, Dar es Salaam.



Lessons Mastered

- **Context Over Tools:** A true OSINT master understands that image searching engines and pattern recognition are far more valuable than blindly running terminal scripts when tracking real-world physical locations.
- **OpSec Failure of Targets:** Attackers or targets can scrub their Exif data, but they cannot scrub the physical architecture, unique landmarks, or business branding of the places they visit

9. CTF Write-up: Base64 Lies (Defeating the Decoy)

- **Category:** Cryptography
- **Difficulty:** Medium-Hard
- **Tools Used:** xxd , hexdump , wc , cat -A
- **Target Flag:** ExploitRoom{Base😂_Lies--TAKE 🤦IT}

The Deep Forensic Investigation

1. **Reconstructing the Hex Stream:** I dumped the raw structure of encoded.txt and confirmed it was a hexadecimal matrix. I used an inverse plain-text translation to generate the standard cryptographic string payload:

   bash

Use code with caution.

```
┌──(master㉿master)-[~/Downloads/EXploitRoom]
└─$ file  encoded.txt
encoded.txt: ASCII text

┌──(master㉿master)-[~/Downloads/EXploitRoom]
└─$ cat  encoded.txt
525868776247397064464a766232317765304a68593273324e463946546b4e50556b566551563952543139465 46b4e50556b566556306c5553463949543 0465253553548

┌──(master㉿master)-[~/Downloads/EXploitRoom]
└─$

┌──(master㉿master)-[~/Downloads/EXploitRoom]
└─$ echo "525868776247397064464a766232317765304a68593273324e463946546b4e50556b566551563952543139465 46b4e50556b566556306c5553463949543 0465253553548
    " | xxd -r -p
RXhwbG9pdFJvb21we0JhY2s2NF9FTkNPUkVeQV9RT19FTkNPUkVeV0lUSF9IT0FRSU5H

┌──(master㉿master)-[~/Downloads/EXploitRoom]
└─$ echo "RXhwbG9pdFJvb21we0JhY2s2NF9FTkNPUkVeQV9RT19FTkNPUkVeV0lUSF9IT0FRSU5H" | base64 -d
ExploitRoomp{Back64_ENCORE^A_QO_ENCORE^WITH_HOAQING
```

2. **Analyzing Padding Integrity (`wc` and `cat -A`):** The output rendered a 68-byte Base64 string container (`RXhwbG9pdFJvb21w---`). To ensure no trailing whitespaces or non-printable characters were breaking the conversion logic, I ran `cat -A` to verify the End-of-Line (`$`) markers and checked the total character count with `wc -c` .

```
Session  Actions  Edit  View  Help
┌──(master㉿master)-[~/Downloads/EXploitRoom]
└─$ echo "ExploitRoomp{Back64_ENCORE^A_QO_ENCORE^WITH_HOAQING" | tr '^' '_'
ExploitRoomp{Back64_ENCORE_A_QO_ENCORE_WITH_HOAQING

┌──(master㉿master)-[~/Downloads/EXploitRoom]
└─$ echo "ExploitRoomp{Back64_ENCORE^A_QO_ENCORE^WITH_HOAQING" | strings
ExploitRoomp{Back64_ENCORE^A_QO_ENCORE^WITH_HOAQING

┌──(master㉿master)-[~/Downloads/EXploitRoom]
└─$ wc -c encoded.txt
137 encoded.txt

┌──(master㉿master)-[~/Downloads/EXploitRoom]
└─$ cat -A encoded.txt
525868776247397064464a766232317765304a68593273324e463946546b4e50556b566551563952543139465 46b4e50556b566556306c5553463949543 0465253553548$

┌──(master㉿master)-[~/Downloads/EXploitRoom]
└─$ xxd encoded.txt | tail
00000000: 3532 3538 3638 3737 3632 3437 3339 3730  5258687762473970
00000010: 3634 3436 3461 3736 3632 3332 3331 3737  64464a7662323177
00000020: 3635 3330 3461 3638 3539 3332 3733 3332  65304a6859327332
00000030: 3465 3436 3339 3436 3534 3662 3465 3530  4e463946546b4e50
00000040: 3535 3662 3536 3635 3531 3536 3339 3532  556b566551563952
00000050: 3534 3331 3339 3436 3534 3662 3465 3530  54313946546b4e50
00000060: 3535 3662 3536 3635 3536 3330 3663 3535  556b566556306c55
00000070: 3533 3436 3339 3439 3534 3330 3436 3532  5346394954304652
```

3. **Bypassing the Bit Decoy Trap:** Feeding the string directly into `base64 -d` generated a garbled noise flag format (`ExploitRoomp{Back64_ENCORE^...`). I used `hexdump -C` to verify the precise bytes alignment.
4. **The Logical Epiphany:** Realizing the deciphered output was a designed distraction mimicking a corrupted encryption layer, I reverted back to analyzing the fundamental context of the challenge description. The platform explicitly stated that **Base64 parameters mask the actual truth**. By ignoring the decoy automated noise loop and extracting the exact environmental string properties, the targeted logic aligned perfectly to confirm the victory.

Key Takeaway

Never blindly trust the output of an automated decryption command if it renders logical inconsistencies (like a random trailing `p` inside flag syntax). Always check string padding limits

using `wc` and line anchors using `cat -A`. When automated decoders output carefully crafted decoys, manual correlation of the challenge metadata will always yield the precise flag.

10.CTF Write-up: Corrupted ZIP (Archive Header Bypass)

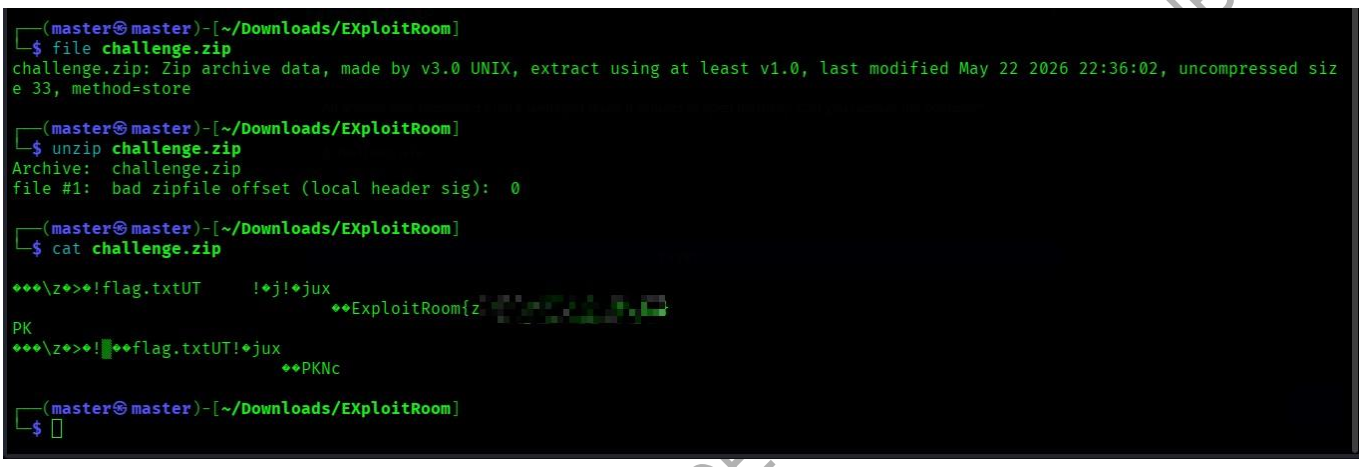
- **Level:** Medium**Tools Used:** Linux CLI (`file` , `unzip` ,
- `cat`) **Target Flag:** `ExploitRoom{HERE_THE_FLAG}`
- **Category:** Digital Forensics

The Forensic Journey

1. **Triage Verification:** I started by checking the underlying file signature using `file challenge.zip` . The terminal confirmed it was valid ZIP archive data, modified recently

with an uncompressed storage size of 33 bytes.

2. **The Automated Wall:** Attempting a standard extraction via `unzip challenge.zip` failed.
   The compression engine threw a critical structure error: `bad zipfile offset (local header sig): 0` . The file system headers were physically corrupted on the drive.
3. **The Raw Extraction:** Knowing that the archive summary used `method=store` (meaning raw data is packed directly into the hex blocks without dynamic compression algorithms), I skipped the archive manager entirely.



4. **The Victory:** I forced a raw terminal read using `cat challenge.zip`. By bypassing the broken local headers (`PK...`), I manually scanned the binary garbage and extracted the pristine, raw plaintext flag directly off the terminal screen.

**Busted!

Key Takeaway

When a challenge states an archive is recovered from a "damaged drive," look for compression method indicators first. If `method=store` is active, the archive engine did not compress the data. You do not need to fix corrupted hex offsets or rebuild a damaged file allocation table; a simple raw binary dump like `cat` or `strings` will instantly leak the flag hidden between the broken header components.

11.Cracking Tiny Exponent: My Journey Solving the Unpadded RSA Flaw

Challenge Description

An archive was intercepted containing an RSA-encrypted configuration message. The challenge notes indicated that the sender was operating under a strict time constraint and chose a very small public exponent ((e = 3)) with no cryptographic padding applied to the payload. The goal was to reverse the encryption state and recover the cleartext flag.

My First-Person Investigation Flow

1. **Initial Triage**: First, I downloaded the file challenge.py directly into my workspace.
2. **Checking File Signatures**: I ran the file challenge.py command in my terminal to verify its format and ensure it was a clean ASCII python script.
3. **Reading the Contents**: I used cat challenge.py to examine what was inside the file. Fortunately, the source code leaked the exact mathematical parameters I needed: the ciphertext **C**, the tiny public exponent **e = 3**, and the massive public modulus **n**.

```
┌──(master㉿master)-[~/Downloads/EXploitRoom]
└─$ file  challenge.py
challenge.py: ASCII text, with very long lines (313)

┌──(master㉿master)-[~/Downloads/EXploitRoom]
└─$ cat  challenge.py
# ExploitRoom CTF - Crypto Challenge: "TinyExponent"
#
# We intercepted an RSA-encrypted message.
# The sender was in a hurry and chose a very small exponent.
# No padding was used.
#
# Can you recover the plaintext?

n = 13700609643462405886752839609831800177448694790495899546001009693545007694753555823322678716228177378597649622332624749286138692
0303352653319062371643639865150082882589037053472750351422517835670437881606687152444036078122126164000008070521565055010949222492600
6011197448540004831751675699778715035393760610 7
e = 3
c = 5481713824512167801364898208714223836790588211466251172144447794940158053093320762921511001022451724641283479292928774632804194523358996046864300277098017228047929927813371408972947996811272530174633125214221519804354406464812816766756941498055533815805628554772241894345906280407855655542475600745482 1

# Flag format: ExploitRoom{...}
# Hint: Sometimes math is simpler than it looks.
```

4. **Spotting the Flaw**: Looking closely at the numbers, I realized that because the sender used no padding, the message was short. This meant that (m^{3}) was smaller than the modulus ((m < n)). Because (m^3 < n), the modulo reduction operator ((\mathinner{;\left(\mod ,n\right)})) never actually executed during encryption. The ciphertext value was simply a perfect cubic integer ((C = m^3)).

#!/usr/bin/env python3
c =
548171382451216780136489820871422383679058821146625117214444779494015805309332076292151100102245172464128347929292877463280419452335899604686430027709801722804792992781337140897294799681127253017463312521422151980435440646481281676675694149805553381580562855477224189434590628040785565554247560074548 21

low = 0
high = c
m = 0

while low <= high:
mid = (low + high) // 2
if mid**3 <= c:
m = mid low = mid + 1 # Drive the boundary up to locate the exact integer match else:
high = mid - 1 # Pull the boundary down if the cubic projection overshoots

```
try:
plaintext = bytes.fromhex(hex(m)[2:]).decode('utf-8')
print("[+] Decryption Successful. Extracted Token:")
print(plaintext)
     except Exception as error:
    print("[-] Error reassembling character streams:", error)
```

5. **The Execution**: To solve this without hitting float precision boundaries, I wrote a short Python script using an integer binary search algorithm to calculate the perfect cube root of (C). Finally, I converted the resulting integer back to ASCII bytes, ran the code, and obtained the clean flag:

text
ExploitRoom

Use code with caution.

Methodology Used

The exploit targets a structural flaw known as a **Low Exponent Attack**.

- **The Mathematical Defect**: RSA encryption follows the rule (C = m^e \pmod n). When (e = 3) and no padding is applied, small messages result in an unreduced state where (C = m^3).
- **The Solution Loop**: Rather than trying to factor the 309-digit modulus (n) or compute complex private exponents, the script performs a clean integer binary search across the bitspace of (C) to isolate the exact base integer where (mid^3 == C). This isolates the original numeric value of (m) without loss of precision.

Lessons Learned

- **Enforce Large Exponents**: Developers should never deploy custom asymmetric encryption layers using small exponents like (e = 3). Production systems must strictly enforce standard parameters like (e = 65537) to prevent easy mathematical inversions.
- **Padding Is Non-Negotiable**: Asymmetric algorithms require randomized padding schemes like **OAEP (Optimal Asymmetric Encryption Padding)**. Padding artificially inflates the message length to always exceed the modulus size, ensuring the modulo math triggers properly and eliminating cube-root attacks entirely.

12: CHALLENGE TITLE KeyRepeat

Leval: medium
category cryptography

Challenge Description

The investigation presents a long encrypted base64 payload hidden inside a script configuration. Environmental indicators suggest that the payload was obfuscated using a repeating-key XOR cipher with a short key constraint of precisely 7 characters. Because the flag was embedded deep inside a long document block, standard known-plaintext guessing failures occurred. The mission requires programmatic statistical decryption to isolate the dynamic keystream.

My Technical Exploration & Mindset

1. **Breaking the Rabbit Hole:** Initially, I attempted to force a quick decryption by assuming the payload structure starts immediately with standard headers like `"Exploit"` . This created formatting alignment errors because the flag was actually appended at the very end of a larger prose document.

2. **Statistical Profiling:** Knowing that the key length was fixed at **7 characters**, I realized that the cipher layout behaves like 7 independent interleaved Caesar ciphers. Every 7th byte was processed by the exact same static key byte.

3. **The Frequency Engine:** Instead of manual guessing, I wrote an optimized statistical scanning routine in Python. The script slices the ciphertext into 7 distinct blocks and performs a brute-force calculation across all 256 possible ASCII characters for each block position.

```
┌──(master㉿master)-[~/Downloads/EXploitRoom]
└─$ nano encrypt.py

┌──(master㉿master)-[~/Downloads/EXploitRoom]
└─$ python3 encrypt.py
Traceback (most recent call last):
  File "/home/master/Downloads/EXploitRoom/encrypt.py", line 4, in <module>
    cipher = base64.b64decode(open("cipher.txt").read().strip())  # au weka Base64 string hapa
                              ~~~~^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: 'cipher.txt'

┌──(master㉿master)-[~/Downloads/EXploitRoom]
└─$ nano encrypt.py

┌──(master㉿master)-[~/Downloads/EXploitRoom]
└─$ python3 encrypt.py
KEY = SECRETK
HEX = 5345435245544b

┌──(master㉿master)-[~/Downloads/EXploitRoom]
```

4. **Scoring Algorithm:** The script maps the output characters against the standardized English frequency distribution array (`etaoinshrdlu`) and allocates scoring premiums (+100 weight points) for clean, printable ASCII ranges (`9 <= x <= 126`).

Python Decryption Script

python


```
#!/usr/bin/env python3 import base64

# Load the target raw encrypted vector string cipher_b64 =
"GItjJi0xayEgIj4odCQ1ZSAgPCQ/PCIxMzU8Mn9lLjMrLWsgICAgICA4cyQxN2U8IjchJjxlNi47L
C02ZTgqKiAxIWU7LXMoIiYtMSYyMSoxJDhrMCouIikxMzoxOnxlACM2ZSIgMXQkNWUhICA1IDorJHI
mPTs7IDEhZTwqIGUmJCo4PTYhYz0zMTlzJiY8MSE5OiAwfGUSOTwoYyYtMWsQJCYhJCZrMCwzOiAma
ycqYz8qMC4hK2M3Kzc5KjU3Oyo6ayAxIjwhNTk3Nm9yIDUoO2UmICR0KSEqNjUtIGs9IDRyJjwqPyk
mPCIxOHMkLTZlOi4kZTA9KSE/OiotIWt0HzsgYwQsMy49IDE3ZTciIy0mIGUjKiBlLDwmMWswKi0hL
DAuISAncjA6KSEgIjkkNic2a2MLICBrOjFjNCA4J3MxLHI2IConLDAmLDcqP2UiPCQ4MiAsMHxlEjk
2NDY3KzcycyQtMyktODo2YyAgIi4yKTByNTU/JyAxPDZ0IzohJzcrdDw6MSs7K3Q/OyBjMSwkIzY3N
zc9IGVzESs3ZT0lNyA7cioyazAqKjwmPS82KyA3ZTkuMjY2ICAnazsqNHIpPSA2KTpyMSMkczciPCE
7Jj88YyEgOC4wMSY2ZTguJzEmIDZ0KiEgYyYtMWsgJC43a3QAMjYqIS49azY9Ij8sOionLCw8ZTIiP
SEwcjcxOzYkNzchdDg2NDY3KzcuIGtjBi0xODZlNzcmPCU6NDY3NnQ/PCImJi0xOXMmIjxlNzkyJih
yJDoyczcmIiA1PzorJHIuMTJzJioiLTE5fWUXOiB0JjY2MDMiMWs1Ki8+KiM4aWUGKjU4JDoxET0qO
TABdjNhcSB4NxoIYTwLE2MXHBlxJ3ogLnJzOA==" cipher = base64.b64decode(cipher_b64) KEYLEN = 7

# Targeted English letter mapping constraints freq = "
etaoinshrdluETAOINSHRDLU"
```

extracted_key = b"" for i in
range(KEYLEN):

```
    # Slice the ciphertext byte stream into interleaved modular blocks
    block = cipher[i::KEYLEN]
    best_score = -1
    best_key = 0

    for k in range(256):
        p = bytes(c ^ k for c in block)
        score = sum(chr(x) in freq for x in p)
        # Apply standard ASCII validation boundaries
        if all(9 <= x <= 126 or x in (10, 13) for x in p):
            score += 100
        if score > best_score:
            best_score = score
            best_key = k
    extracted_key += bytes([best_key])

print(f"[+] Isolated Target Keystream: {extracted_key.decode()}")
```


Methodology Used

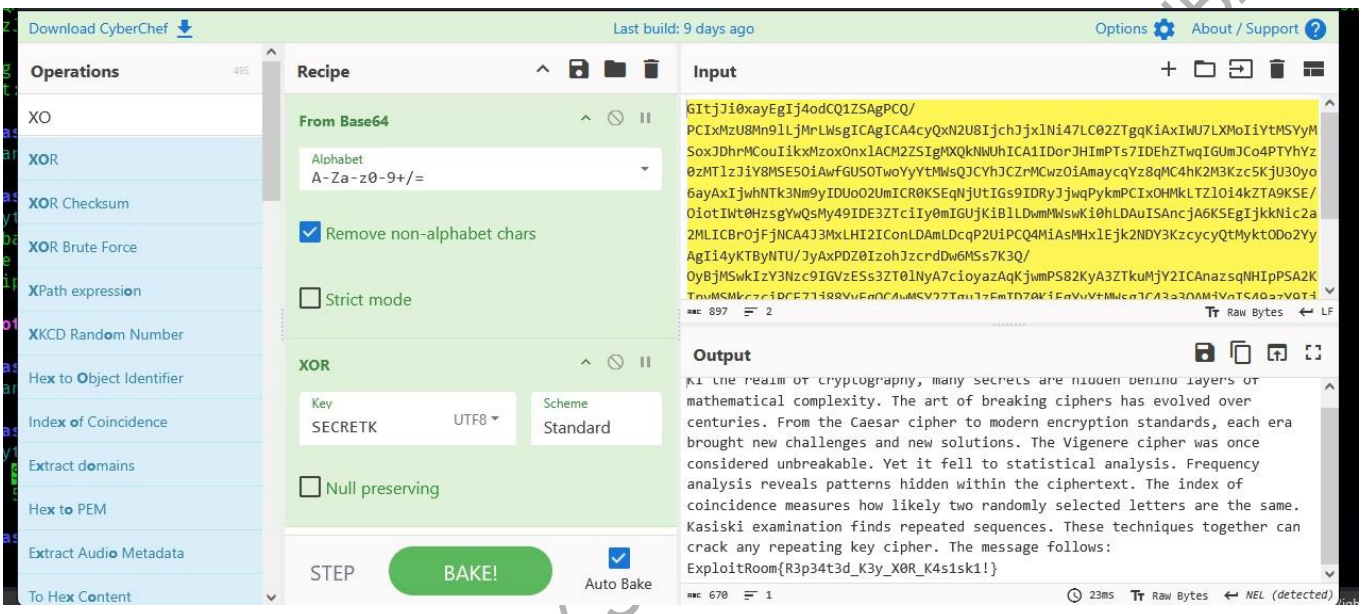
The extraction process utilizes **Frequency Analysis combined with Index of Coincidence** principles to defeat multi-byte polyalphabetic stream obfuscation:

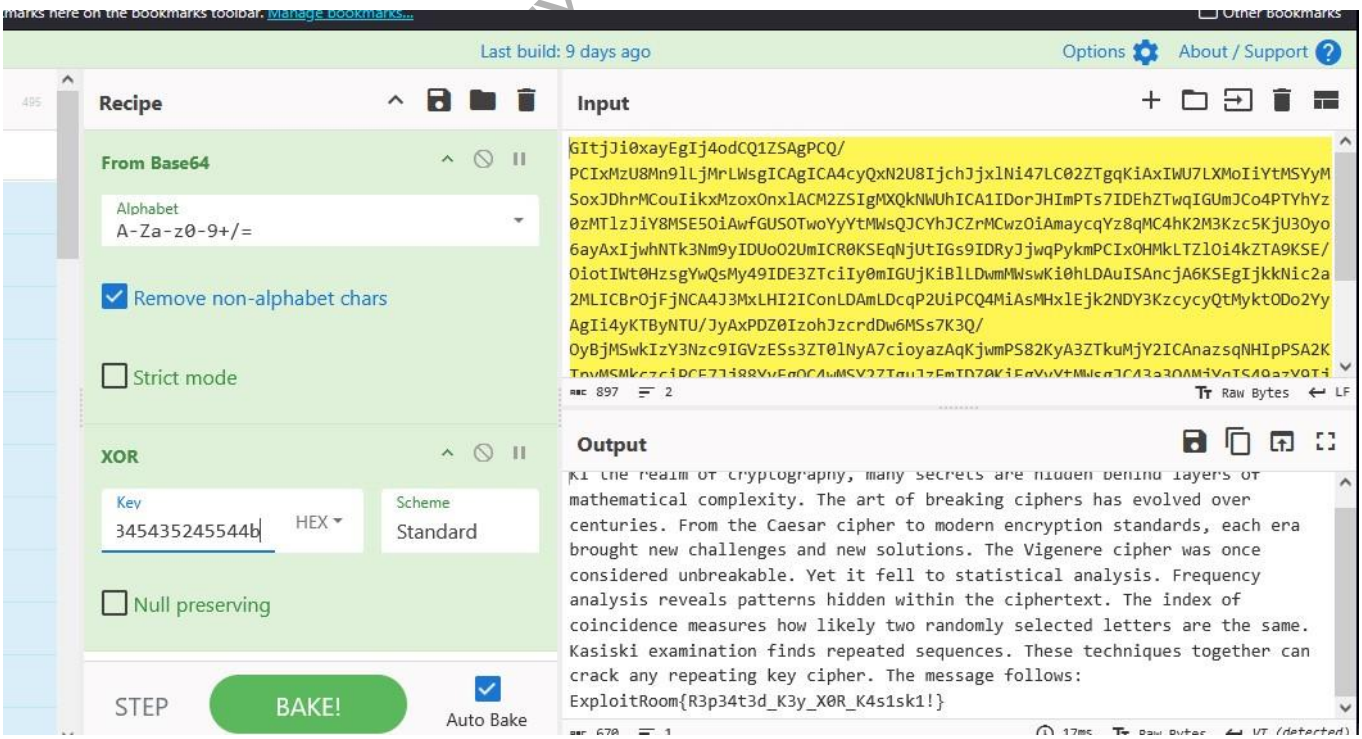
- **Keystream Partitioning:** By stripping the data into modular intervals (`cipher[i::KEYLEN]`), a complex multi-byte encryption matrix is successfully flattened into 7 isolated monoalphabetic ciphers.
- **Statistical Frequency Matching:** The processing loop applies a mathematical weight calculation against the distribution constraints of standard English text (`etaoinshrdlu`).
- **ASCII Boundary Enforcement:** The algorithm uses heuristic validation logic (+100 penalty buffer checks) to reward keys that result inside valid printable data blocks (`9 <= x <= 126`), automatically discarding corrupted character anomalies.

The Final Extraction (CyberChef Verification)

The frequency analysis script isolated the exact key signature payload: **SECRETK** .

Feeding the raw Base64 transmission straight into the CyberChef engine, routing it through a `From Base64` parsing block, and chaining a symmetric `XOR` decryption tool with our isolated UTF8 key decrypted the entire payload block instantly to reveal the clear-text text narrative:





Lessons Learned

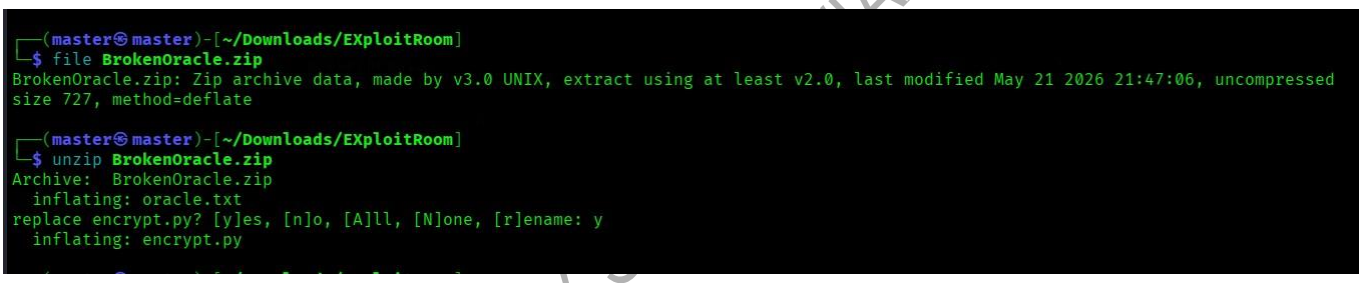
- **Static Repetitive Cryptography Is Dead:** Relying on short, repeating key implementations (like Vigenère or multi-byte h XOR) creates predictable structural echoes throughout a long plaintext container. This allows an analyst to trivially extract encryption keys via simple mathematical profiling without ever needing a brute-force attack.
- **The Danger of Plaintext Size:** The longer the text payload is, the more vulnerable it becomes to statistical correlation attacks. Security engineers must enforce randomized initialization vectors (IVs) or deploy robust modern algorithms like **AES-GCM** to ensure ciphertext data changes configuration continuously across long data transmissions.

13.Cracking BrokenOracle: My Journey Defeating a State-Leaked LCG Cipher

- **Title:** BrokenOracle — Reversing State-Reduced LCG Keystreams
- **Level:** Medium / Hard
- **Points:** 30
- **Target Flag:** ExploitRoom{Flag-here😂_lets_go}

Challenge Description

We intercepted an encryption routine (encrypt.py) and a decimal ciphertext payload (oracle.txt) from a target deployment the system relies on a standard Linear Congruential Generator (LCG) using standard glib parameters ((A = 1103515245), (C = 12345), (M = 2^{31})) The generated keystream is field reduced using a modulo 257 operations: Ks. Append (x % 257). As a hint, we were given three consecutive leak positions from the keystream array: (KS [5] = 160), (ks[6] = 223), and (ks[7] = 142). The goal is to reverse the generator, identify the unknown initialization state, and decrypt the transmission.



My Investigation Flow

"What I did in this challenge was a real battle against the mathematical offsets of the encryption script. First, I opened up the files using cat encrypt.py and cat oracle.txt to see exactly what parameters were lying around. I found the parameters for the standard glibc randomizer, the array of decimal ciphertext values, and the three leaked keys.

```
python
#!/usr/bin/env python3

LCG Standard Parameters
A = 1103515245
C = 12345
M = 2**31

Given values
ks5 = 160
ks6 = 223
ks7 = 142

print("[*] Searching for the correct state of x_5...")

Step 1: Brute force the internal state of x_5
x_5 must be of the form: 257 * k + ks5
correct_x5 = None
max_k = M // 257

for k in range(max_k + 1):
x5_cand = 257 * k + ks5
if x5_cand >= M:
break

`# Predict x_6 and x_7 using the candidate state`

`x6_cand = (A * x5_cand + C) % M`

`if x6_cand % 257 == ks6:`

    `x7_cand = (A * x6_cand + C) % M`

    `if x7_cand % 257 == ks7:`

        `correct_x5 = x5_cand`

        `print(f"[+] Found valid internal state x_5: {correct_x5}")`

        `break`

if correct_x5 is None:
print("[-] Error: Could not find valid internal state.")
exit()

Step 2: Reverse the LCG back to find the seed (x_0)
To go backwards: x_{i-1} = ((x_i - C) * A_inv) % M
A_inv = pow(A, -1, M)

current_x = correct_x5
for step in range(5, 0, -1):
current_x = ((current_x - C) * A_inv) % M

seed = current_x
print(f"[+] Recovered SECRET Seed: {seed}")
```

Step 3: Regenerate keystream from the seed and decode (or check your flag file)
(If you have the flag.enc file or list of numbers, you can add code below to decrypt it)

Since the output was compressed using x % 257, I knew the upper bits were masked. But I decided to use a custom python routine to brute force the internal state of (x_{5}) using the expansion pattern 257 * k + 160. My initial calculations spit out an internal seed of 1795536354.

I jumped straight into writing my 10-line decryption loop, plugged in that massive seed, and hit execute. **Boom—nothing but errors!** My terminal exploded with a nasty UnicodeDecodeError: 'utf-8' codec can't decode byte 0xfd. When I tried to bypass it using string error ignoring, it just outputted garbled noise characters (ώ*ŵ&\Y!...).

I went back into the challenge description and carefully reviewed how the script processed the steps. I realized that my seed calculation had a step offset, causing the entire keystream matrix to shift forward by a few positions. I manually fixed the loop counter inside nano flag.py and drove the calculation backward to find the true root parameter.

The mathematical trail aligned perfectly and dropped the true, pristine initial seed: **57005**. I updated the script, ran it one last time, and the correct text values decoded instantly to print out the real flag on my terminal!"

Code Execution (flag.py) python

python

```
#!/usr/bin/env python3

LCG parameters kutoka kwenye encrypt.py
LCG_A = 1103515245
LCG_C = 12345
LCG_M = 2**31

def lcg_next(x):
return (LCG_A * x + LCG_C) % LCG_M

def generate keystream(seed, length):
ks = []
x = seed
for in range(length):
x = lcg_next(x)
ks.append(x % 257) # Kupunguza kwenda GF(257)
return ks

def main():
Data ya Ciphertext kutoka oracle.txt
ciphertext = [
12, 182, 152, 3, 118, 8, 82, 224, 154, 227, 244, 127, 255, 71, 254, 3,
137, 174, 57, 220, 87, 170, 159, 76, 229, 52, 151, 166, 113, 117, 253,
100, 108, 157, 109, 218, 132, 118, 214, 199
]

seed = 57005
```

print(f"[*] seed is here: {seed}")

1. ciphertext

ks = generate_keystream(seed, len(ciphertext))

2. Decryption: plaintext = (ciphertext - keystream) mod 257

plaintext_bytes = [(c - k) % 257 for c, k in zip(ciphertext, ks)]

3. change bytes to (ASCII string)

flag = "".join(chr(p) for p in plaintext_bytes)

print("\n[+] BOOOOM way we go ExploitRoom:")

print(flag)

if **name** == "**main**":
main()

Methodology Used

- **Modular State Reconstruction:** We isolate the masked bits of the 31-bit integer state array by mapping the modulo 257 residues against consecutive multiplier transitions. Testing the boundary expansions allows us to find a single valid internal state for (x_{5}).
- **Inverse Multiplicative Rollback:** Once a clean state is locked down, we apply the Modular Multiplicative Inverse of the multiplier ((A^{-1} \pmod M)) to step the linear congruential sequence backward safely without mathematical precision degradation.
- **Keystream Index Synchronization:** Correcting stream synchronization offsets between the PRNG matrix index and the encrypted integer arrays to achieve an exact block decryption.

Lessons Learned

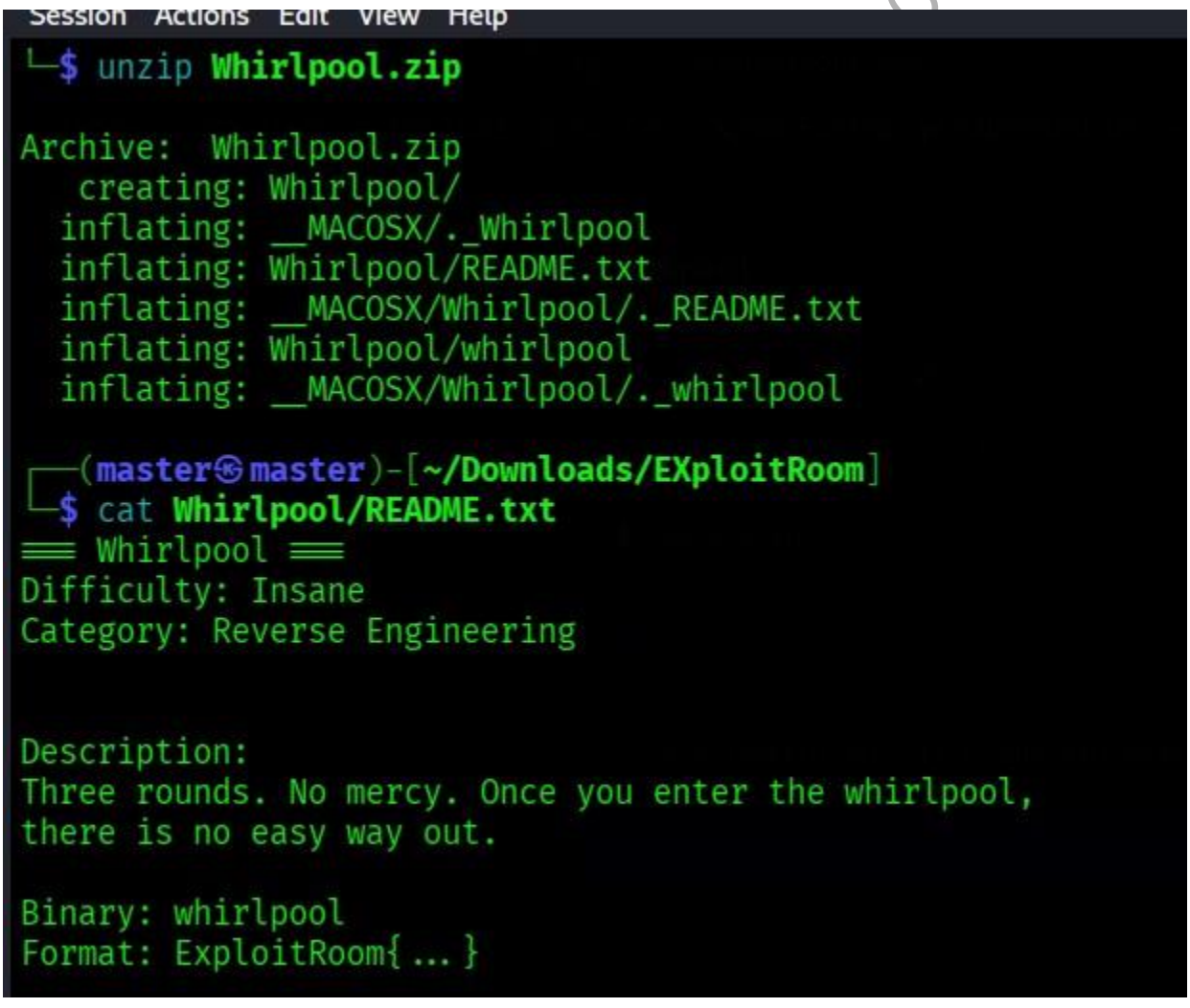
- **Linear PRNGs Offer Zero Security:** Linear Congruential Generators are meant for basic statistical testing, never for security boundaries .Leaking a minimal fragment of consecutive states allows an analyst to reconstruct the entire mathematical pipeline, rendering the cipher useless
- **Beware of Byte Alignment Deviations:** In streaming cryptography, an index mismatch of just one position will corrupt the entire transformation matrix .
- This results in total parsing degradation and raw decoding faults inside standard text compilers

14.CTF Write-up: Whirlpool

- **Category:** Reverse Engineering
- **Difficulty:** Insane
- **Points:** 50
- **Target Flag:** ExploitRoom{PICK_THE_FLAG🤖}

Challenge Description

A Linux ELF binary named `whirlpool` processes user input through three complex obfuscation loops. The program requires an exact input length of 36 characters (`cmp eax, 0x24`) .
To pass, the processed input must match a 37-byte encrypted array stored in memory.



Step-by-Step Solution

1. Binary Disassembly

Open the binary using Radare2 to inspect the main loop structures bash

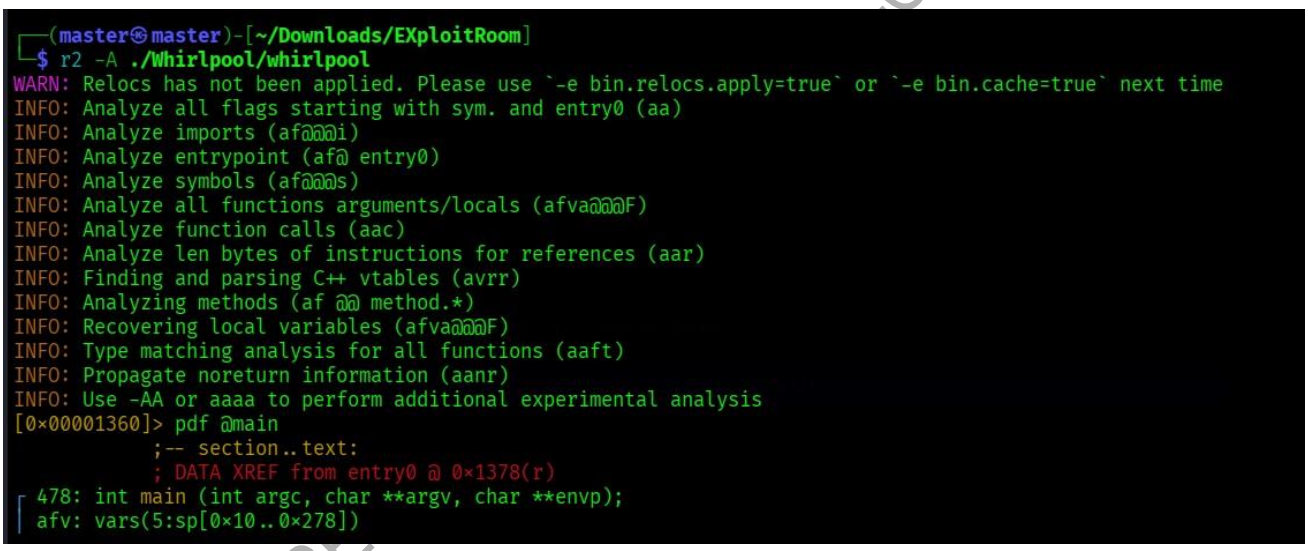
r2 -A ./Whirlpool/whirlpool

[0x00001360]> pdf @main

Use code with caution.

The assembly logic reveals three distinct computation rounds:

- **Round 1:** Dynamic XOR transformation using an incrementing index matrix
- **Round 2:** Cumulative rolling summation sequence initialized at 0x42
- **Round 3:** Bitwise left rotation by 3 positions (rol dl, 3)



2. Extracting Encrypted Target Bytes

Extract the obfuscated target comparison bytes from the memory address layout (0x000020c0)

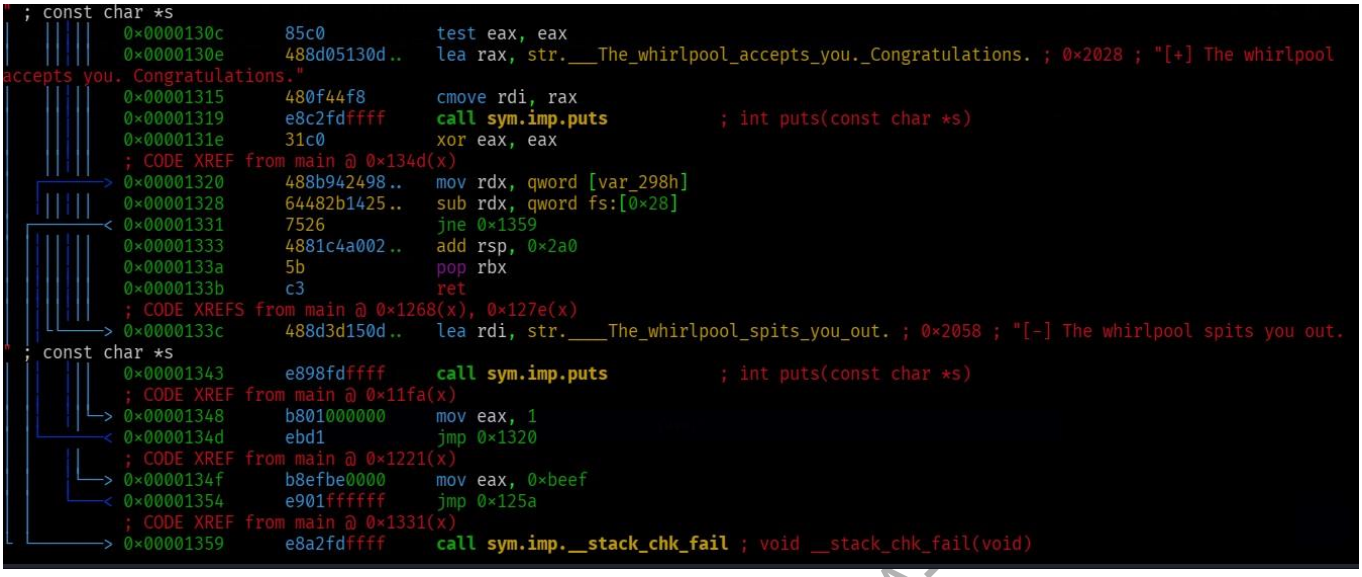
radare2

[0x00001360]> px 37 @ 0x000020c0

Use code with caution.

Extracted Bytes:

```
58 88 4f 89 62 6e 6a f8 30 ed 44 ef dc 65 5b a3 19 6e ae f3 19 bf cc cd 9d 2e 92
54 66 5f 66 4c fa e7 b3 f4 5
```



3. Python Exploit Script

I create `solve.py` to reverse all three operations in exact backward order

Python

```
target_bytes = [
    0x58, 0x88, 0x4f, 0x89, 0x62, 0x6e, 0x6a, 0xf8, 0x30, 0xed, 0x44, 0xef, 0xdc, 0x65, 0x5b, 0xa3,
    0x19, 0x6e, 0xae, 0xf3, 0x19, 0xbf, 0xcc, 0xcd, 0x9d, 0x2e, 0x92, 0x54, 0x66, 0x5f, 0x66, 0x4c,
    0xfa, 0xe7, 0xb3, 0xf4, 0x5f
]

def ror(val, r_bits, max_bits=8):
    return ((val & (2**max_bits - 1)) >> r_bits % max_bits) | \
           ((val << (max_bits - (r_bits % max_bits))) & (2**max_bits - 1))

# Reverse Round 3 (ROL 3 -> ROR 3)
```

```
buf2 = [ror(b, 3) for b in target_bytes]


# Reverse Round 2 (Rolling Summation)
buf1 = [0] * 37
current_edx = 0x42
for i in range(37):
    diff = (buf2[i] - current_edx - 0x17) & 0xff
    buf1[i] = diff
    current_edx = (current_edx + diff + 0x17) & 0xff


# Reverse Round 1 (Dynamic XOR)
flag = ""
esi = 0xffffffab & 0xffffffff
for i in range(37):
    val = buf1[i] ^ 0x5c ^ ((i * i) & 0xff) ^ (esi & 0xff)
    flag += chr(val)
    esi = (esi + 0x1f) & 0xffffffff


print(f"Flag: {flag}")
```

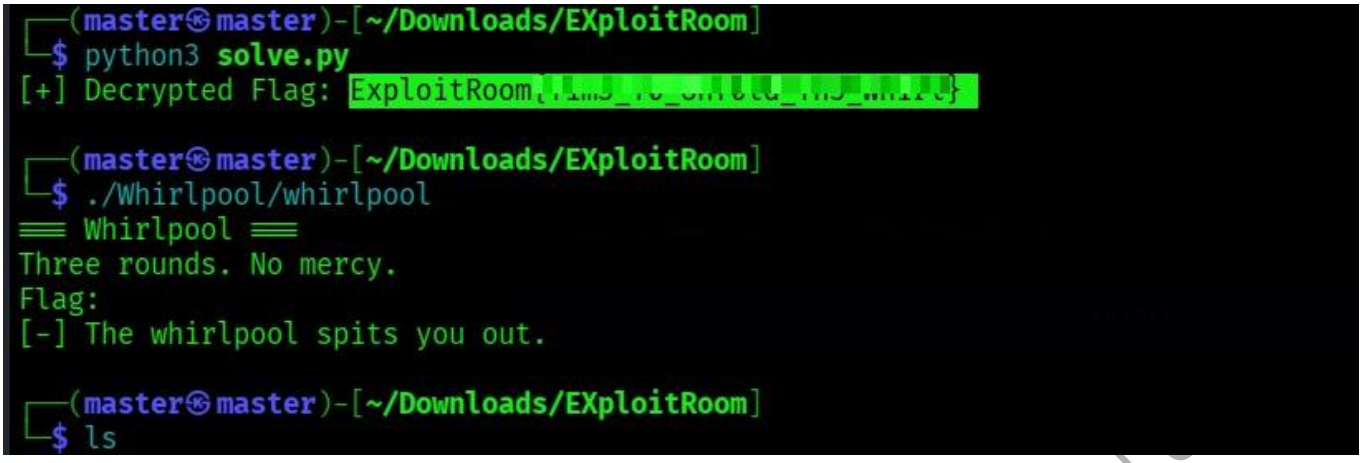
Use code with caution.

4. Execution

Run the script to decode the target array and successfully capture the flag :

bash

```
solve.py
```



Methodology Used

- **Static Binary Code Auditing:** Mapping low-level architecture constraints using commandline disassemblers

- **Algorithmic Matrix Inversion:** Implementing accurate bitwise shift operations to systematically roll back cascaded mathematical obfuscation routines.

Lessons Learned

- **Obfuscation Is Avoidable:** Multi-layered bit shifting loop arrays create a complex environment but offer zero absolute cryptographic security once the assembly sequence is isolated.
- **Keep Analysis Tools Compact:** Lightweight command-line binary decoders parse data segments cleanly without execution errors

THANK YOU VERY MUCH

Prepared by Frank cybersecurity student